

A small app, on your iPhone

A practical guide to logging, keeping your data safe, and understanding how a website becomes a personal app.

Daily Check-in is a progressive web app (PWA): an interactive website that you can add to your Home Screen. You do not compile anything on your iPhone. GitHub builds the code and serves the app; your phone runs it.

Install once, then open the icon

1. Open [Daily Check-in in Safari](#) while connected to the internet.
2. Tap **Share**, then **Add to Home Screen**. If available, enable **Open as Web App**. Tap **Add**.
3. Open the new **Daily Check-in** icon. Let the first online load finish so the app can prepare its offline files.
4. Use the installed app consistently. Safari and an installed web app may use different data containers.

No MacBook needed

For this app, a browser, GitHub repository, and hosting are enough. The app installs from a web link rather than the App Store.

What is included

One editable daily record for perceived intake, weight, and optional cardio. Trends help you review what you have logged. History lets you revisit earlier dates. Settings provides CSV export, JSON backup/restore, and an optional cloud account for syncing between devices.

App: [Open Daily Check-in](#)

Source and version history: [GitHub repository](#)

Installation reference: [Apple: Turn a website into an app in Safari](#)

A check-in takes a moment

The date defaults to today in Hong Kong time. You can record a missed day, but future dates are not accepted.

Your daily fields

Perceived intake (1-5) is your impression: very low, low, usual, high, or very high. It is not a calorie calculation. **Weight** is entered in kilograms.

Cardio: choose Yes or No. If Yes, enter your own cardio type, minutes, and burned calories. The type can be walking, cycling, swimming, or any text that makes sense to you. Numbers are manually entered; the app does not measure activity or calculate calories.

Example for 6 October

Intake: 3 (usual) | Weight: 70.2 kg
Cardio: Yes | Type: Walking
Minutes: 30 | Burned calories: 180 kcal

The 180 kcal value is an example input, not an estimate supplied by this app.

Saving and correcting

Tap **Save check-in**. A date has one record: saving it again replaces that date's existing entry. Use **History** → **Edit** to change it. Blank fields mean not recorded; an explicit cardio No means you recorded no cardio.

You can save an intake, weight, or cardio answer without filling every field. A cardio No can therefore be a useful check-in on its own. Previous records remain valid after the cardio update.

Reading trends

Use the selected time range to review changes and logged days. Missing observations stay missing rather than becoming zero. The weight average uses the available weights within seven calendar days. Cardio summaries reflect only your entered values, so partial logs are a partial picture.

Where does the journal live?

The public code repository and your daily records are two different things. Choose local-only use or an optional cloud account.

When you tap Save, JavaScript validates the entry and writes it to **IndexedDB**, a database managed by your browser on your phone. The database is called **daily-check-in**. It keeps the device journal, separate account records, and settings. Reopening normally reloads these records and draws the graphs.

Guest: a separate device journal

Without signing in, entries stay local. They are not uploaded to GitHub or Supabase. Your existing logs remain in this guest journal after the cloud update. Signing in does not upload them automatically or merge them into another account.

Signed in: local plus cloud

Entries saved while signed in are kept locally, then sent to your Supabase account when online. Sign in to that same account on another device to retrieve successfully synced entries. GitHub serves public app code; it does not receive the journal records.

Two copies, when sync succeeds

Your phone has a working local copy. Supabase has the synced account copy. Guest entries and pending offline edits still have only the local copy until you explicitly copy or sync them.

Privacy and storage boundaries

Account access is protected by login and database rules that restrict each user to their own records. The app does not add its own encryption to journal content. An unlocked phone or trusted web code at the same origin may access local records; keep control of your account and deployed code.

An origin means protocol + hostname + port. The URL path is not a security boundary. Clearing browser storage can remove local data. Private browsing is unsuitable for a lasting journal. Keep JSON backups outside the app.

References: [MDN: Using IndexedDB](#); [MDN: Storage quotas and eviction](#)

Sign in, then choose your data

An account is optional. You can keep using the app locally if cloud accounts are unavailable or you prefer local-only use.

Create an account or sign in

In **Settings** → **Your cloud account**, choose **Create account**, enter your email and a password of at least six characters, then submit. Use a strong, unique password. If asked, open the email confirmation link; return to the app and sign in afterward.

For an existing account, enter your email and password and tap **Sign in**. Your account journal appears separately from the guest device journal. New saves while signed in belong to this account.

Bring your previous device entries along

First export a JSON backup of the guest journal. After signing in, use **Copy device journal to my account**, review the count, and choose **Confirm copy**. The guest journal is retained. You need a connection so the app can check the account before copying.

If any date is already present in the account, the copy stops instead of silently replacing it. Review those dates manually. A deliberate JSON restore can replace matching dates after its preview and confirmation; keep a backup before doing so.

Forgotten password

Use the password reset option with your account email. Open the recovery email link and follow **Choose a new password** → **Update password**. Email confirmation and password recovery depend on the project email service, not the offline cache.

Current email delivery limitation

The default Supabase sender is for testing: recipients must be project organization team addresses, and it currently limits the project to two emails per hour. Other addresses require a configured custom SMTP service. A missing email may therefore be a service restriction, not an incorrect password.

Reference: [Supabase: Auth email and custom SMTP](#)

Know when the cloud is safe

Saving locally and successfully syncing to the server are separate steps.

Check the status in Settings

While signed in, look for **Synced, 0 pending changes**, and no conflicts. **Sync now** pulls account records and sends pending changes while online. The app also attempts sync when you return to it or regain a connection. It is not a continuously running background service.

Before clearing data or changing phone

Connect to the internet, tap Sync now, and confirm there are no pending changes or unresolved conflicts. Export a JSON backup too. On the other device, sign in to the same account and sync to recover the server copy.

Successfully synchronized records can be retrieved after browser data is cleared, as long as the account and cloud service remain available. **Unsynced offline edits and guest-only entries cannot be recovered from the cloud.** Clearing storage first can lose that work. The app blocks normal sign-out while changes or conflicts remain.

If two devices change the same date

The app shows both versions instead of silently overwriting one. Compare **This device** with **Cloud**, then choose **Use this device** or **Use cloud**. That choice keeps one full record for the date; it does not combine fields. A deletion may be one of the versions.

Sync is not a historical backup

Deleting an entry while signed in propagates the deletion to your account and other devices. Cloud sync gives continuity across devices, but it is not an independent archive of every previous version. Continue exporting JSON backups and CSV files periodically.

Cloud journal sync and push reminders are separate services. The reminder service uses push subscriptions and reminder settings; its delivery setup has not been verified complete. Synced journal entries do not mean reminders are working.

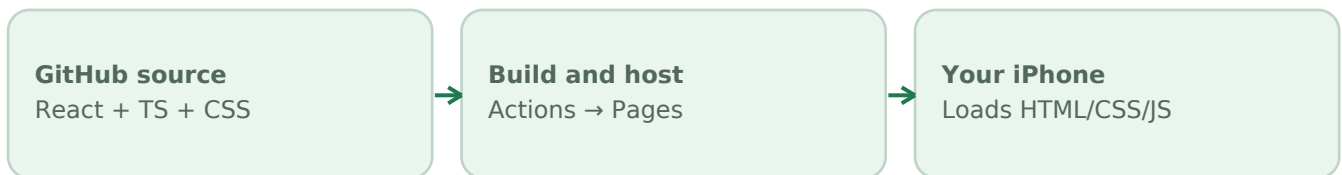
Access-control reference: [Supabase: Row Level Security](#)

How HTML becomes an app

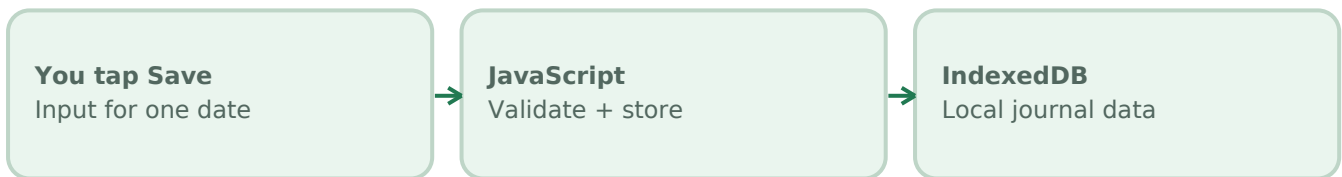
HTML supplies the starting document; JavaScript supplies the interactive behavior.

HTML is the page skeleton and loads the app files. **CSS** controls colors, spacing, and responsive layout. **JavaScript** handles input, saving, exporting, and charts. This app uses React to construct the interface; Vite converts its source into deployable static files.

The two flows



ONLINE DELIVERY: files travel to the phone. Journal records are not sent to GitHub in this flow.



LOCAL SAVE: phone storage. Signed-in entries also sync to Supabase when online; guest entries do not.

Why can it run offline?

A **service worker** is a browser-managed helper. After a successful online installation, this app caches its HTML, JavaScript, CSS, icons, and fonts. Later it can load those files without downloading them again. This code cache is separate from the IndexedDB journal.

Offline use depends on that initial cache being ready and retained. The service worker is not a program that can run continuously in the background; the browser controls when it starts and stops.

Reference: [MDN: Offline and background operation](#)

Protect your entries

Choose the right export for what you want to do next.

CSV: for spreadsheets and analysis

In **Settings** → **Export spreadsheet (CSV)**, save or share the file using the iPhone share sheet. Open it in Numbers, Excel, or Google Sheets. Each row is a date; columns include intake level, weight, cardio Y/N, type, minutes, and calories. Empty cells mean not recorded.

CSV is an analysis export. This app does not restore from CSV. If you edit spreadsheet values, those changes do not automatically flow back into your journal.

JSON: for backup and restoration

Use **Settings** → **Export backup** and save the JSON file to Files, iCloud Drive, or another safe location. This backup contains the currently viewed guest or account entries, not your password, reminder subscription, or settings. Keep dated copies, especially before deleting data, replacing your phone, or changing the app address.

On the destination phone/app, choose **Restore from backup**, select the JSON file, review the preview, and confirm. Incoming dates replace matching dates; other dates remain. Version 1 backups from the older app are still accepted. New version 2 backups include cardio fields.

A useful routine

Once a week: export a JSON backup.
Before a risky change: export another backup.
For charts outside the app: export CSV too.
Check that Files really contains the saved files.

How the app updates

A push to GitHub's main branch runs checks, builds the files, and deploys them to Pages. When you next go online, your browser can discover the new service worker. Save your current form, close all app/Safari windows for this app, and reopen to let the waiting update activate.

An update at the same address does not intentionally erase the local journal. The cloud release keeps previous guest records and stores account records separately. Still, backup first: hosting changes and browser storage loss need explicit recovery from your JSON file.

Is this a useful way forward?

Yes - for small personal tools with forms, local records, exports, and charts, a PWA is a practical approach.

You can reuse the same pattern for a habit diary, expense log, reading tracker, symptom notes, or study planner: define a record, build an interface, save it locally, add export/restore, then host the static files. You can develop on Windows or Linux and install through Safari without a Mac or App Store release.

Choose the approach around the features

Need	This PWA approach	Native iOS app
Manual logs and charts	Good fit; offline storage	Also suitable
Install and update	Web link + Home Screen	Apple build/distribution process
Cross-device journal sync	Available with Supabase login	Needs a sync design too
Apple Health / continuous tracking	Not provided by this app	Native APIs may be needed

This app does not read Apple Health, GPS workouts, or heart rate, and does not continuously measure exercise in the background. Burned calories are your input. Reliable scheduled push reminders require the separate backend plus device permission and testing; a cached page alone cannot guarantee a daily alert.

A simple development recipe

1. Start with the smallest useful form and one clear data model.
2. Decide whether local-only data is enough; add backups early.
3. Test saving, reopening, offline use, and restoration on the actual phone.
4. Publish a version, keep its history in GitHub, and make small updates.

Further reading: [MDN: What is a progressive web app?](#)

This guide describes the project implementation at v1.2.0. iPhone menus and browser capabilities can vary with iOS versions. Use the linked primary documentation for current platform guidance.